

产品接口文档

一、接入必读

1.1 适用范围

本文档的适用对象为接入平台第三方产品技术人员、日常维护人员。

1.2 合作公司须知

2. 合作公司在接入时，需提供外网ip，以便将贵公司ip放入白名单
3. 合作公司在接入时，需要先申请开通公司账号

1.3 接口说明

通信协议

- 接口请求方式为 **POST**
- 采用 HTTPS 协议进行请求和响应的处理，接口请求及响应返回均为 JSON 格式的数据
- 接口未注明处均使用 UTF-8 编码

注意事项

以下使用的个人数据纯属虚构，仅表示格式，如有雷同，纯属巧合

响应时间

接口超时时间为 **4 秒**

签名

调用时需根据 body 中的参数以及商务提供的 appId 和 secret 对参数进行签名处理，参考 [3.1 加签方式](#)

环境说明

- **正式环境**：正式账号使用，调用已开通接口无限制，按成功条数收费
- **测试环境**：测试账号使用，调用已开通接口返回挡板数据
- **账号说明**：正式账号仅适用于正式环境，测试账号仅适用于测试环境

1.4 请求结构

参数名称	示例值	类型	是否必填	参数说明
appld	y89098io	String	是	appld 【由商务分配】
sign		String	是	签名
apiKey	产品编号	String	是	固定：idRiskTagV107
encryptionType	2	int	否	参数加密类型 1：明文【默认】 2：md5
body		Json	是	接口请求体，见各接口详细定义

1.5 响应结构

参数名称	示例值	类型	参数说明
code	0	Number	响应编码 0:成功
msg	成功	String	返回文字描述
seqNo	vu6pjf4g0400	String	交易流水号
data		Object	接口数据体，见各接口详细定义
- busiCode	10	int	业务响应 10:查得数据，计费标识
- busiMsg	查询成功	String	业务响应消息
- result		JSON	响应内容

1.6 全局返回码 code 说明

code 返回码	说明
0	成功
-1	响应异常

busiCode 返回码	说明
10	查询成功【计费】
1000	数据未查得【计费】
1001	账户余额不足
1002	账户信息不存在
1003	appld 异常
1004	产品编号异常
1005	账号信息异常
1006	透支余额已达上限
1007	数据请求异常
1009	服务尚未开通

二、身份风险V107

2.1 接口地址

https://{域名}/enol/api/v1/doCheck

- 测试域名：testenol.cn
- 生产域名：enol.com.cn

2.2 请求参数

body 参数说明

参数名称	示例值	类型	是否必填	参数说明
name	张三	String	是	姓名
idCard	310000199001010010	String	是	身份证
tradeNo	457fa900b705291ada6	String	否	业务单号

请求示例

```
{
  "encryptionType": 1,
  "appId": "y89098io",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "apiKey": "idRiskTagV107",
  "body": {
    "name": "张三",
    "idCard": "310000199001010010",
    "tradeNo": "457fa900b705291ada6"
  }
}
```

2.3 响应内容

响应示例

```
{
  "code": 0,
  "msg": "请求成功",
  "data": {
    "busiCode": 10,
    "busiMsg": "success",
    "result": {
      "detail": [
        "A0"
      ]
    }
  },
  "seqNo": "48u8tgsvwd60"
}
```

风险类型说明

参数名称	参数说明
A	涉案人员
B	其他前科
C	涉赌人员
D	涉性人员
E	交通肇事
F	肇事肇祸
G	妨害社会管理秩序
H	危害公共安全
I	金融套现
J	金融诈骗
K	侵犯财产
L	制贩毒人员
M	涉黑涉恐
N	严重刑事
O	危害国家安全
P	在逃人员

风险周期代码说明

参数名称	参数说明
0(只存在A0)	无风险
1	无可回溯日期
2	1年内
3	1-3年
4	3-5年
5	5-10年
6	10年及以上

三、加签

3.1 签名方式

1. **筛选并排序**: 获取所有请求的业务参数【body 中的参数】，不包括字节类型参数，如文件、字节流，不包含值为空的参数，并按照第一个字符的键值 ASCII 码递增排序（字母升序排序），如果遇到相同字符则按照第二个字符的键值 ASCII 码递增排序，以此类推。
2. **拼接**: 将排序后的参数与其对应值，组合成"参数参数值"的格式连接起来，最后将 secret 拼接到末尾，此时生成的字符串为待签名字符串。如假设业务参数为 idCard, mobile, tradeNo【具体按接口规定的业务参数为准】，并都有相应的值，则待签名字符串为：

```
idCardce62d2fdd5564508386384fda0cee40fmobile2ea01442a7eebf816aef379c74c0fa53tradeNo457fa900b705291ada6secret
```

3. **调用签名函数**: 使用各自语言对应 MD5 进行签名。
4. **赋值**: 把生成的签名赋值给 sign 参数，加入到请求参数中。

3.2 Demo

本demo所使用的http库为Apache HttpClient5, json库为alibaba fastjson2

demo中的appId和appSecret替换为贵司的，建议配置在配置文件或数据库中

```
import com.alibaba.fastjson2.JSONObject;
import lombok.extern.slf4j.Slf4j;
import org.apache.http.client5.http.classic.methods.HttpPost;
import org.apache.http.client5.http.impl.classic.CloseableHttpClient;
import org.apache.http.client5.http.impl.classic.HttpClients;
import org.apache.http.core5.http.ContentType;
import org.apache.http.core5.http.io.entity.EntityUtils;
import org.apache.http.core5.http.io.entity.StringEntity;
import java.security.MessageDigest;
import java.util.Map;
import java.util.stream.Collectors;

@Slf4j
public class Demo {

    public static void main(String[] args) {
        String url = "https://api.enolfax.com/enol/api/v1/doCheck";
        String appId = "";
        String appSecret = "";
        String apiKey = "loanRiskTagV23_2_1"; // 产品编码，具体取值参考产品文档
        Integer encryptionType = 2; // 加密类型: 1:明文 2:md5 3:sha256 4:sm3

        // 业务参数，具体参数请参考对应的产品文档
        JSONObject body = new JSONObject();
        body.put("idCard", "ce62d2fdd5564508386384fda0cee40f");
        body.put("mobile", "2ea01442a7eebf816aef379c74c0fa53");
        body.put("tradeNo", "457fa900b705291ada6");
        // ...

        // 拼接签名参数
        String signParams = body.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .map(e -> e.getKey() + e.getValue())
            .collect(Collectors.joining())
```

```

        + appSecret;
        String sign = genMd5(signParams);

        // 构建请求参数
        JSONObject reqParams = new JSONObject();
        reqParams.put("appId", appId);
        reqParams.put("apiKey", apiKey);
        reqParams.put("sign", sign);
        reqParams.put("encryptionType", encryptionType);
        reqParams.put("body", body);

        // 开始请求
        String resp = post(url, reqParams.toJSONString());
        log.info("resp->{}", resp);
    }

    // post请求
    public static String post(String url, String jsonBody) {
        String result = null;
        HttpPost httpPost = new HttpPost(url);
        httpPost.setEntity(new StringEntity(jsonBody, ContentType.APPLICATION_JSON));

        try (CloseableHttpClient httpClient = HttpClients.createDefault()) {
            result = httpClient.execute(httpPost, response -> {
                // 获取响应信息
                return EntityUtils.toString(response.getEntity());
            });
        } catch (Exception e) {
            log.error("post请求失败, url->{}, jsonBody->{}", url, jsonBody, e);
        }
        return result;
    }

    // md5加密
    public static String genMd5(String text) {
        try {
            byte[] buffer = MessageDigest.getInstance("md5").digest(text.getBytes());
            StringBuilder sb = new StringBuilder();
            for (byte b : buffer) {
                int a = b & 0xff;
                String hex = Integer.toHexString(a);
                if (hex.length() == 1) {
                    hex = "0" + hex;
                }
                sb.append(hex);
            }
            return sb.toString();
        } catch (Exception e) {
            throw new RuntimeException("md5 error");
        }
    }
}

```